

Sorting and Patterns - Day 5

Binary-search boundaries and integrated practice

Priyanka's Platinum study plan | Friday, September 11, 2026 | 120 minutes | C11 and Python 3

Today's question: what proves that discarded candidates cannot contain the answer? Prerequisites: sorted-array pointer reasoning, loop invariants, Day 4's heapsort, and integer division. Begin in Python, then translate to C. The earlier binary-search module is useful retrieval material; today's focus is deriving boundaries and composing them with other algorithms.

Your two-hour session

Minutes	Task	Deliverable
0-15	Derive lower/upper bound	Eliminated-region invariants
15-40	Lab A: bounds and occurrence range, both languages	Empty/absent/duplicate tests
40-65	Lab B: integer square root, both languages	Monotone answer predicate
65-95	Lab C: unique 3Sum, both languages	Reused heapsort and duplicate skipping
95-110	Debug four short fragments	Smallest failing input and repair
110-120	Closed-notes synthesis	Six-sort table and one proof

Attempt each block before viewing the appendix. Use the supplied C output callback and the previous heapsort to keep the 3Sum block focused on its algorithm, not result-buffer plumbing. If stuck, annotate and repair the reference within the allocated block.

Lab A - Search for a boundary, not just equality

Given a sorted integer array, `lower_bound` returns the first index whose value is $\geq$ target, or `n` if no such element exists. `upper_bound` returns the first index whose value is $>$ target, or `n`. Neither function mutates input. Examples on `[1,2,2,2,5]`: target 2 $\rightarrow$ lower 1, upper 4; target 3 $\rightarrow$ both 4; target 8 $\rightarrow$ both 5.

A linear scan works, but sorted order means the predicate $a[i] \geq \text{target}$ changes from false to true at most once. Use `[lo,hi)` for unclassified array positions. Lower-bound invariant: all positions before `lo` have values $<$ target; all positions at or after `hi` have values $\geq$ target. The unknown boundary itself lies in the CLOSED range `[lo,hi]`, so `n` remains a valid answer even though it is not an array index.

When $a[\text{mid}] < \text{target}$, sorted order rules out every position through `mid`, so $\text{lo} = \text{mid} + 1$. Otherwise `mid` might be the first valid index, so $\text{hi} = \text{mid}$. In both cases $\text{hi} - \text{lo}$ decreases. At $\text{lo} = \text{hi}$, the eliminated-region invariant identifies the boundary. Changing the comparison to $\leq$ derives upper bound: the false region now contains values $\leq$ target.

Implement both functions and a wrapper returning first occurrence, last occurrence, and count. Python uses `(-1,-1,0)` for absence. C returns count 0 and writes `n` into both output indices for absence, avoiding a negative sentinel in `size_t`. Check the count before indexing either output.

Test `[]`, `[2]`, `[2,2,2]`, `[1,2,2,2,5]`, targets below, within, between, and above values. First/last occurrence must remain $O(\log n)$; do not find one match then scan a long duplicate run. Derive $\text{count} = \text{upper} - \text{lower}$ and $\text{last} = \text{upper} - 1$ only when $\text{count} > 0$. In C compute $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$ to avoid overflowing `lo + hi`.

Lab B - Binary search over possible answers

Return $\text{floor}(\sqrt{N})$ for integer $N \geq 0$ without a square-root library. For C use 32-bit int input $0 \leq N \leq \text{INT_MAX}$. Examples: $0 \rightarrow 0$, $1 \rightarrow 1$, $8 \rightarrow 2$, $9 \rightarrow 3$, $2147483647 \rightarrow 46340$.

Direct approach: try every integer through the answer, taking $O(\sqrt{N}+1)$ time. Instead candidate x is feasible when $x^2 \leq N$. Feasibility is true for a prefix of nonnegative integers and false thereafter. Search the inclusive candidate interval $[lo, hi] = [0, N]$, retaining the largest feasible candidate seen as answer.

Invariant: candidates below lo are feasible; candidates above hi are infeasible; answer stores the greatest tested feasible value. For a feasible mid , record it and discard candidates through mid by setting $lo = mid + 1$. For an infeasible mid , set $hi = mid - 1$. The active interval strictly shrinks; when empty, answer is the last feasible integer.

Avoid overflow in $mid * mid$ by testing $mid == 0$ OR $mid \leq N / mid$. Integer division is appropriate for nonnegative values. Short-circuit the zero case to avoid division by zero. Use long long C search bounds so $mid + 1$ is representable even at the input limit.

Tests: $0, 1, 2, 3, 4, 8, 9, 15, 16, 17, \text{INT_MAX}$. Verify the mathematical postcondition $rr \leq N < (r+1)(r+1)$ with widened C arithmetic in the test harness. Give complexity in terms of numeric value N , then in terms of its bit length under the unit-cost arithmetic model.

Lab C - Unique 3Sum from a previous sort

Given integer array a , return every distinct VALUE triplet (x, y, z) with $x \leq y \leq z$ and $x + y + z = 0$, using three distinct input indices. Duplicate occurrences may be used if present. Mutating the input by sorting is allowed. Example $[-1, 0, 1, 2, -1, -4] \rightarrow [(-1, -1, 2), (-1, 0, 1)]$. $[0, 0, 0, 0] \rightarrow [(0, 0, 0)]$ exactly once. No solution yields an empty output.

Brute force tries all index triples, $O(n^3)$, then still needs deduplication. Reuse Day 4's heapsort. Fix i and search the suffix with $left = i + 1$, $right = n - 1$. For a fixed i , the target pair sum is $-a[i]$. The two-pointer discard proof from Day 1 applies unchanged. Bounds $i < left < right$ guarantee distinct indices.

Skip a fixed value if it equals the previous fixed value. After emitting a match, skip all occurrences of the matched left and right VALUES within the active interval. Explain why for fixed $a[i]$ and $a[left]$, the required right value is uniquely determined, so skipping repeated endpoint values loses no distinct triplet.

Python returns a list of tuples. C streams each triplet to `emit(x, y, z, context)`; the callback can print or store it and must not mutate the array. This avoids a hidden fixed output-capacity limit. The callback typedef and reference are supplied; implement the core loop. Link C with `day04.c`; keep `day04.py` beside `day05.py` for the Python import.

Tests: $[], [0], [0, 0], [0, 0, 0, 0], [-1, 0, 1, 2, -1, -4], [1, 2, 3], [-2, 0, 0, 2, 2]$, and $[\text{INT_MIN}, 1, \text{INT_MAX}]$. Widen the FIRST C addition, before overflow can happen. Compare output sets and also check that output length equals the size of that set.

Debug clinic - Find a counterexample first

These are intentionally incorrect fragments. Explain the invariant or safety condition each violates, then repair it.

TEXT

- A. lower bound: `if a[mid] < target: lo = mid`
- B. variable window: `if total >= target: record(); remove_left()`
- C. heap extraction: `swap(a[0], a[size-1]); sift_down(a, 0, size)`
- D. cycle loop: `while fast.next: fast = fast.next.next`

Final closed-notes review

For all six sorts, state best, average, worst time, auxiliary space, stability, and the relevant input/implementation assumptions. Distinguish comparisons, writes, and measured runtime. Give one example where the best-case time changes because of an early exit. Explain why a comparison-sorting worst-case lower bound does not forbid linear time on special inputs.

Submit C/Python code, boundary traces, the sqrt postcondition checks, 3Sum duplicate tests, and one initialization-preservation-termination proof. Record the first broken invariant for any unfinished problem, so the next practice session can target it.

Solution appendix

Boundary and answer checks

On [1,2,2,2,5], target 2: lower-bound intervals are [0,5), [0,2), [0,1), then lo=hi=1. Upper-bound intervals are [0,5), [3,5), [3,4), then lo=hi=4. First=1, last=3, count=3. On empty input both bounds return 0 without indexing. Two binary searches still cost $O(\log(n+1))$ total and $O(1)$ workspace.

For $\text{sqrt}(8)$, candidate mids are 4 (infeasible), 1 (feasible), 2 (feasible), 3 (infeasible), so answer=2. Search uses $O(1+\log(N+1))$ time and $O(1)$ auxiliary machine words. If b is the input bit length, this is $O(b)$ iterations; arbitrary-precision division is not unit-cost, so that is not a universal $O(b)$ bit-operation bound for Python.

3Sum cost and repair checks

Sorting costs $O(n \log n)$ worst time with reused heapsort. For each fixed index, the two endpoints together advance $O(n)$ times; across $O(n)$ choices, total time is $O(n^2)$, assuming $O(1)$ work to emit a triplet. Auxiliary workspace excluding output is $O(1)$ with this iterative sort. Python output storage is $O(k)$ for k triplets; C output storage belongs to the callback. Neither answer should silently ignore output cost.

[-2,0,0,2,2] emits only (-2,0,2). [INT_MIN,1,INT_MAX] emits that one triplet on 32-bit int platforms because $-2147483648+1+2147483647=0$. Comparing distinct index triples instead of value triples would create unwanted duplicates.

Debug A fails on [1,2], target 2: mid=0 repeats. Use lo=mid+1. B fails on [1,1,4], target 4: shrinking once misses the one-element answer; use while and update best before removing. C reintroduces the extracted maximum: [2,1] becomes sorted then is heapified back; use size-1. D dereferences null on an empty list, or on a later null fast; guard fast AND fast.next.

Six-sort comparison for these references

Best/worst cases below range over integer arrays permitting duplicates. Average assumes uniformly random permutations of distinct keys. Bounds describe growing n ; empty/singleton work is constant.

Sort	Best time	Average time	Worst time	Extra space	Stable
Bubble, early exit	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes
Selection	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	No
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes
Merge, full merges	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Yes
Quick, two-way Lomuto	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$ balanced; $O(n)$ worst	No
Heap, early-stop sift	$\Theta(n)$ on all equal	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	No

Bubble/insertion require strict greater-than comparisons for the stability stated here. Merge requires left-first ties. Quicksort is in-place at the partition level but still consumes stack space. Heap's duplicate-allowing best case depends on its early-stop sift; do not transfer it without checking the implementation and domain.

The comparison-sorting lower bound is a WORST-case statement: a decision tree must distinguish $n!$ strict orders, giving depth at least $\log_2(n!) = \Omega(n \log n)$. Some paths and special inputs can still be much shorter. Runtime measurements alone cannot prove this lower bound or a best-case formula.

Progressive hints

Bounds: name the false region before choosing an inequality. Sqrt: search the predicate rather than an array, and separate zero from division. 3Sum: derive one fixed-index scan first, then add duplicate skipping. Every fix should have a smallest counterexample and a new passing test.

Python 3 reference functions

PY

```
from day04 import heap_sort
```

PY

```
def lower_bound(a, target):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

PY

```
def upper_bound(a, target):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if a[mid] <= target:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

PY

```
def occurrences(a, target):
    first = lower_bound(a, target)
    end = upper_bound(a, target)
    return (first, end - 1, end - first) if first < end else (-1, -1, 0)
```

PY

```
def integer_sqrt(n):
    # Contract: integer n >= 0.
    lo, hi, answer = 0, n, 0
    while lo <= hi:
        mid = lo + (hi - lo) // 2
        if mid == 0 or mid <= n // mid:
            answer = mid
            lo = mid + 1
        else:
            hi = mid - 1
    return answer
```

PY

```
def three_sum(a):
    # Mutates a; returns distinct value triplets in sorted order.
    heap_sort(a)
    result = []
    for i in range(len(a) - 2):
        if i > 0 and a[i] == a[i - 1]:
            continue
        left, right = i + 1, len(a) - 1
        while left < right:
            total = a[i] + a[left] + a[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append((a[i], a[left], a[right]))
                low, high = a[left], a[right]
                while left < right and a[left] == low:
                    left += 1
                while left < right and a[right] == high:
                    right -= 1
    return result
```

C11 reference functions

```

C
#include <stddef.h>

C
void heap_sort(int *a, size_t n); /* Link with day04.c. */

C
size_t lower_bound(const int *a, size_t n, int target) {
    size_t lo = 0, hi = n;
    while (lo < hi) {
        size_t mid = lo + (hi - lo) / 2;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

C
size_t upper_bound(const int *a, size_t n, int target) {
    size_t lo = 0, hi = n;
    while (lo < hi) {
        size_t mid = lo + (hi - lo) / 2;
        if (a[mid] <= target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

C
size_t occurrences(const int *a, size_t n, int target,
                  size_t *first, size_t *last) {
    size_t begin = lower_bound(a, n, target);
    size_t end = upper_bound(a, n, target);
    *first = begin < end ? begin : n;
    *last = begin < end ? end - 1 : n;
    return end - begin;
}

C
int integer_sqrt(int n) {
    /* Contract: 0 <= n <= INT_MAX, with 32-bit int. */
    long long lo = 0, hi = n, answer = 0;
    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (mid == 0 || mid <= n / mid) {
            answer = mid;
            lo = mid + 1;
        } else hi = mid - 1;
    }
    return (int)answer;
}

C
typedef void (*TripletSink)(int, int, int, void *);

```

C

```
void three_sum(int *a, size_t n, TripletSink emit, void *context) {
    /* emit is non-null; it must not mutate a. */
    heap_sort(a, n);
    if (n < 3) return;
    for (size_t i = 0; i < n - 2; ++i) {
        if (i > 0 && a[i] == a[i - 1]) continue;
        size_t left = i + 1, right = n - 1;
        while (left < right) {
            long long total = (long long)a[i] + a[left] + a[right];
            if (total < 0) ++left;
            else if (total > 0) --right;
            else {
                emit(a[i], a[left], a[right], context);
                int low = a[left], high = a[right];
                while (left < right && a[left] == low) ++left;
                while (left < right && a[right] == high) --right;
            }
        }
    }
}
```